Redis

Read hot data. Analyze it with SQL.

Install & load

```
INSTALL redis FROM community;
LOAD redis;
```

01 At a glance

Query Redis strings, hashes and lists from DuckDB, or snapshot operational data into local tables for analysis. Every Redis call selects a named DuckDB secret, keeping connection details separate from the data-access expression.

Redis keys → SQL functions → Local analysis

02 Useful Redis operations

`redis_get(key, secret)`

Read a string; redis_set writes a key and value.

`redis_hget(key, field, secret)`

Read one hash field; redis_hset writes one and returns the count of fields added.

`redis_hgetall(key, secret)`

Return a table with field and value columns.

`redis_lrange_table(key, start, stop, secret)`

Return list positions and values; stop = -1 includes the end.

`redis_scan(cursor, pattern, count, secret)`

Return next_cursor:keys_csv; repeat until the returned cursor is zero.

`redis_hscan_over_scan(pattern, fields, n, secret)`

Stream key, field and value rows from matching hashes.

`redis_exists(key, secret)`

Check existence; redis_type reports the stored value type.

`redis_expire(key, seconds, secret)`

Set a TTL; redis_expireat takes a Unix timestamp and redis_ttl reads the remaining seconds.

03 Keep in mind

Choose table-shaped results

Use redis_hgetall and redis_lrange_table for relational results. Scalar redis_mget and redis_lrange return comma-separated strings. redis_get and redis_hget return NULL for a missing key or field (wrap them in coalesce for a default); redis_mget keeps an empty entry for each missing key so positions match the requested keys.

04 Name the connection

```
CREATE SECRET quickref_redis (
  TYPE redis,
  host 'localhost',
  port '6379'
);
```

For a local Redis server; add its required password to the secret configuration.

05 Read a hash as rows

REQUIRES SETUP

```
SELECT field, value
FROM redis_hgetall(
  'user:42', 'quickref_redis'
);
```

Requires the server and a Redis hash at user:42.

06 Snapshot matching hashes

REQUIRES SETUP

```
CREATE TABLE users_snapshot AS
SELECT key, field, value
FROM redis_hscan_over_scan(
  'user:*', '*', 100, 'quickref_redis'
);
SELECT key, count(*) AS fields
FROM users_snapshot
GROUP BY key;
```

Use a pattern containing only hashes. Subsequent analysis reads the local snapshot.

07 In practice

Scope scans and connections

Key listing uses cursor-based SCAN, never KEYS, but redis_keys and redis_hscan_over_scan walk every page; narrow the pattern on large keyspaces. A scan is not a transactionally consistent snapshot while Redis changes. The documented transport is single-node plain TCP; configure the connection and network boundary accordingly.

Full docs & examples ↗ Source ↗